

CSCI 210: Computer Architecture

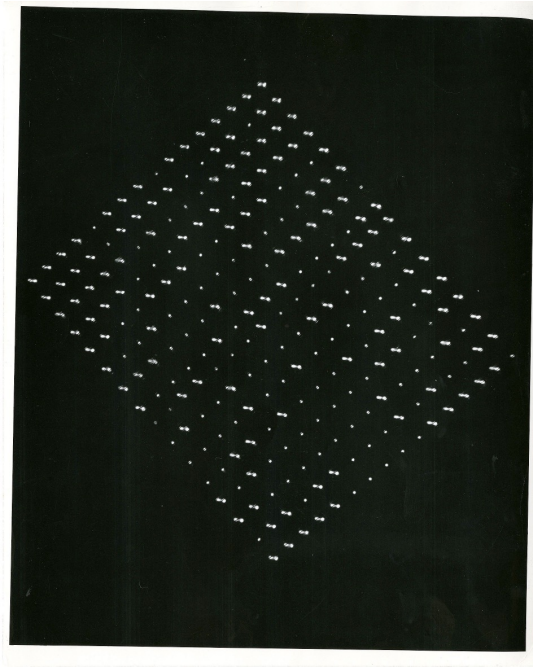
Lecture 34: Caches III

Stephen Checkoway
Slides from Cynthia Taylor

CS History: The Williams Tube



ArnoldReinhold,
CC BY-SA 3.0



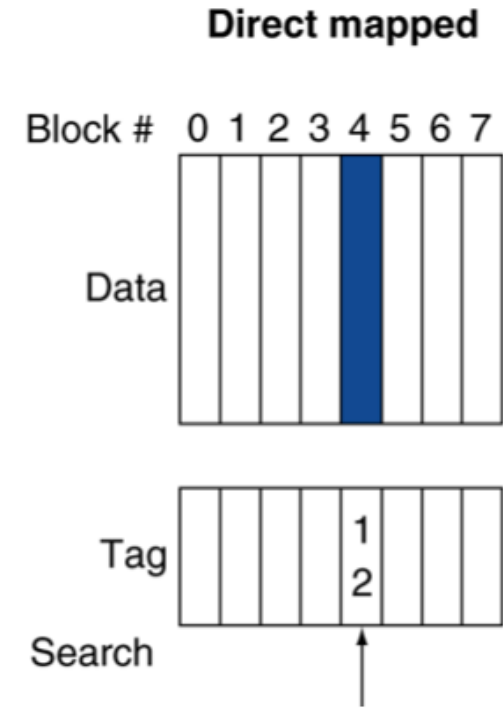
National Institute of
Standards and
Technology, Public
domain, via Wikimedia
Commons

- First random-access storage device
- Developed in 1946
- Displays a grid of dots over a cathode ray tube (using an electron beam to strike phosphor)
- Each dot represents a bit
- Each dot creates a small static electricity charge
- Charge at each location is read by a metal sheet in front of the display
- Needs to be periodically refreshed as charge fades over time

ASSOCIATIVE CACHES

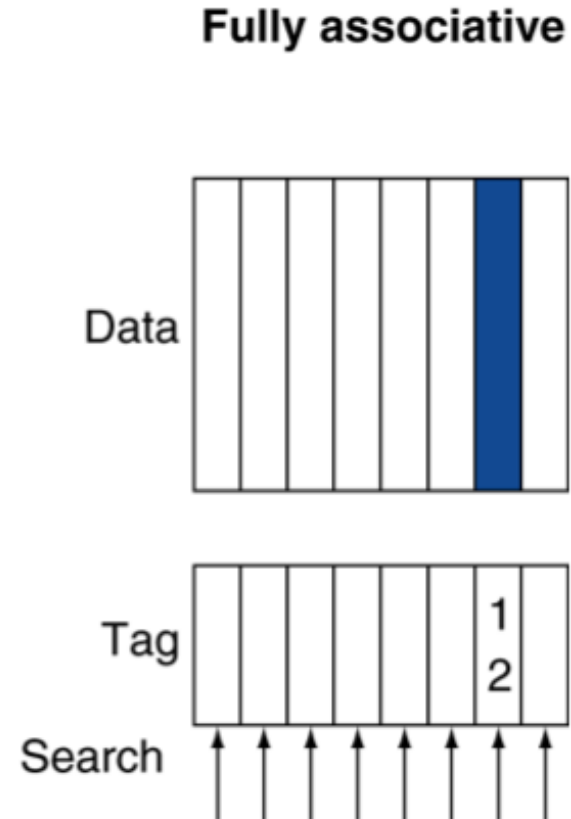
Direct-mapped Cache

- Each block goes into **1** spot
- Only search one entry
- Associativity = 1
- What if we allow blocks to go into more than one spot?



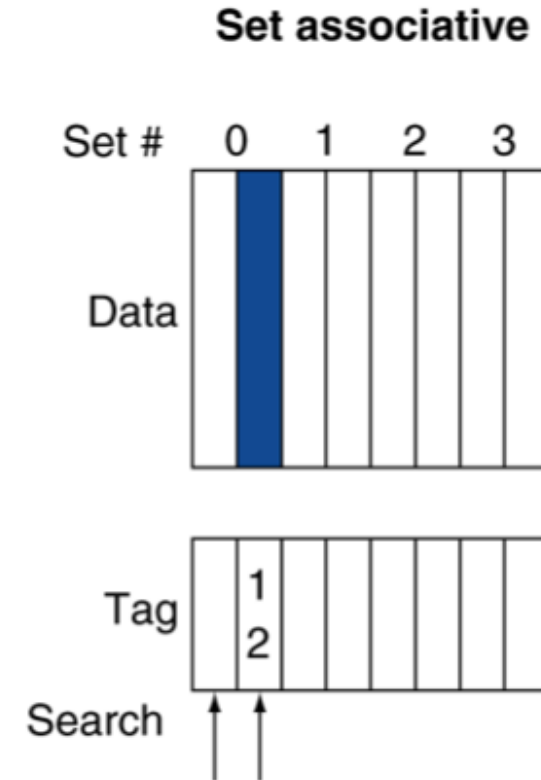
Fully-associative Cache

- Allow a given block to go in any cache entry
- Requires all entries to be searched at once
- Comparator per entry (expensive)



n -way Set-associative Cache

- Each set contains n entries
- Block number determines which set
 - (Block address) modulo (#Sets in cache)
- Search all entries in a given set at once
- n comparators (less expensive than fully associative)



Spectrum of associativity for 8-entry cache

One-way set associative (direct mapped)

Block	Tag	Data
0		
1		
2		
3		
4		
5		
6		
7		

Two-way set associative

Set	Tag	Data	Tag	Data
0				
1				
2				
3				

Four-way set associative

Set	Tag	Data	Tag	Data	Tag	Data	Tag	Data
0								
1								

Eight-way set associative (fully associative)

[illegible]

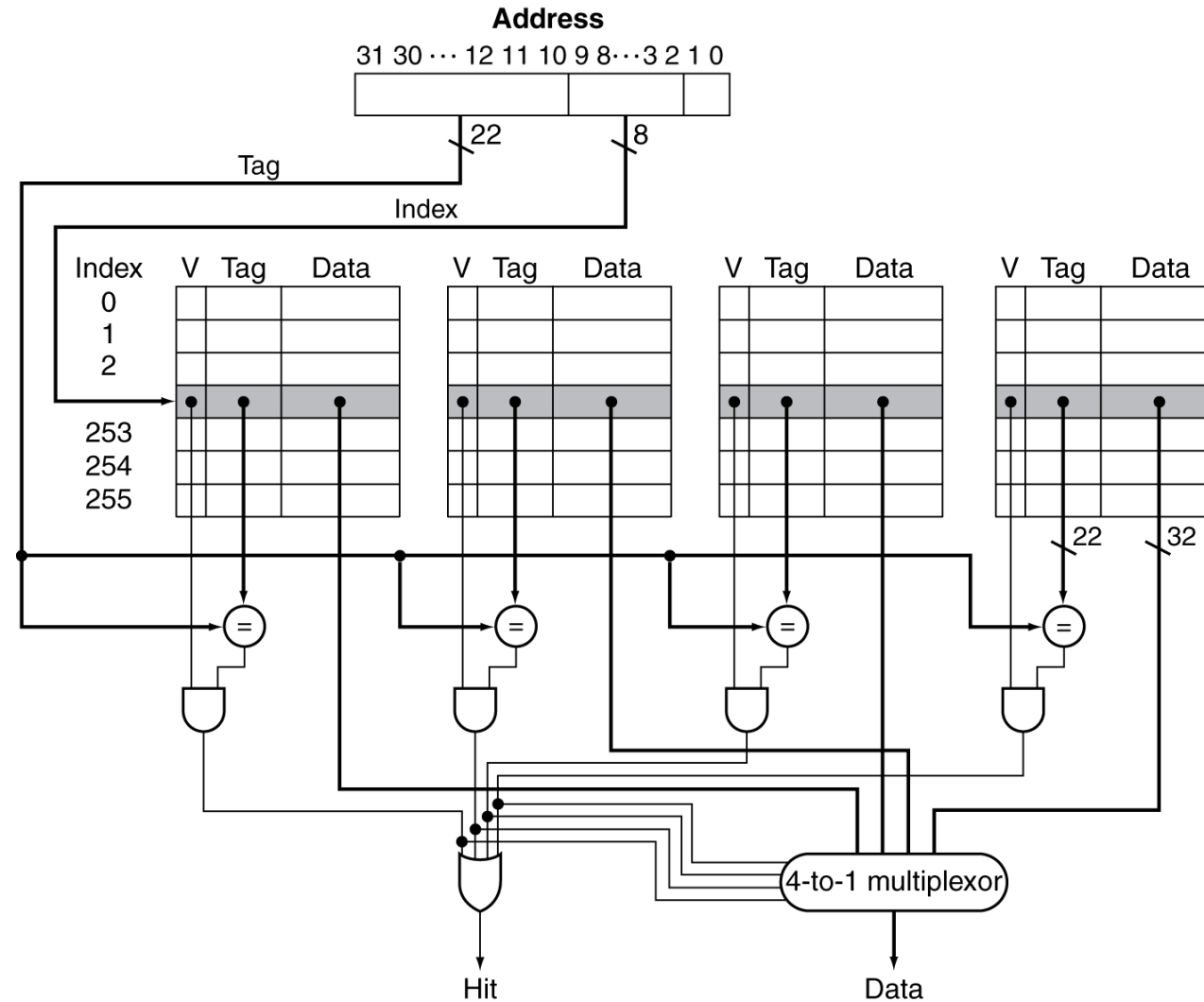
Memory addresses, block addresses, offsets

0 0 0 1 0 1 0 1 1 1 0 0 1 0 0 1 1 0 1 0 1 1 0 0 1 0 1 0 0 1 1

- Block size of 32 bytes (not bits!)
- 16-block, 2-way set associative cache
- Each address
 - A (32 – 5)-bit block address (in purple and blue)
 - A 5-bit offset into the block (in green)
- Block address can be divided into
 - A (32 – 3 – 5)-bit **tag** (purple)
 - A 3-bit cache **index** (blue)

V	Tag	Data	V	Tag	Data
0			0		
0			0		
0			1	3F2084	...
0			0		
0			0		
1	15C9AC	...	1	28477D	...
0			0		
0			0		

Set Associative Cache Organization



Given a 256-entry, **8-way set associative cache** with a block size of 64 bytes, how many bits are in the tag, index, and offset?

	Tag bits	Index bits	Offset bits
A	$32 - 5 - 6 = 21$	5	6
B	$32 - 3 - 5 = 24$	3	5
C	$32 - 8 - 6 = 18$	8	6
D	$32 - 6 - 5 = 21$	6	5
E	$32 - 6 - 3 = 23$	6	3

Given a 256-entry, **fully associative cache** with a block size of 64 bytes, how many bits are in the tag, index, and offset?

	Tag bits	Index bits	Offset bits
A	$32 - 5 - 6 = 21$	1	6
B	$32 - 3 - 5 = 24$	3	5
C	$32 - 8 - 6 = 18$	8	6
D	$32 - 6 - 5 = 21$	6	5
E	$32 - 0 - 6 = 26$	0	6

Replacement Policy

- Direct mapped: no choice
- Set associative
 - Prefer non-valid entry, if there is one
 - Otherwise, choose among entries in the set
 - Goal: Choose an entry we will not use in the future

Replacement Policy

- Least-recently used (LRU)
 - Choose the one unused for the longest time
 - Simple for 2-way, manageable for 4-way, too hard beyond that
- Random
 - Gives approximately the same performance as LRU for high associativity

Associativity Example

- Compare 4-block caches
 - Direct mapped, 2-way set associative, fully associative
 - Block access sequence: 0, 8, 0, 6, 8
- Direct mapped

Block address	Cache index	Hit/miss	Cache content after access			
			0	1	2	3
0	0					
8	0					
0	0					
6	2					
8	0					

Associativity Example: 0, 8, 0, 6, 8

- 2-way set associative

Block address	Cache index	Hit/miss	Cache content after access			
			Set 0		Set 1	
0	0					
8	0					
0	0					
6	0					
8	0					

- Fully associative

Block address		Hit/miss	Cache content after access			
0						
8						
0						
6						
8						

Three types of cache misses

- Compulsory (or cold-start) misses
 - first access to the data.
- Capacity misses
 - we missed only because the cache isn't big enough.
- Conflict misses
 - we missed because the data maps to the same index as other data that forced it out of the cache.

block address of misses

4
8
12
4
8
20
4
8
20
24
12
8
4

tag	data

DM cache

Cache miss example (from StackOverflow)

32 kB direct-mapped cache

1. You repeatedly iterate over a 128 kB array
 - All misses but the first access to each line are capacity misses because the array does not fit in cache; the first are compulsory misses
2. You iterate over two 8 kB arrays that map to the same cache indices
 - These are conflict misses because if you changed the locations of the arrays to be consecutive, then both would fit in the cache

Conflict vs. capacity miss

- A block of memory has been accessed previously (so it was at one point in the cache)
- It is no longer in the cache
- Is it a conflict miss or a capacity miss?
 - If a **fully-associative cache** with the same number of cache entries would have a **miss**, then it's a **capacity miss**
 - If a **fully-associative cache** with the same number of cache entries would have a **hit**, then it's a **conflict miss**

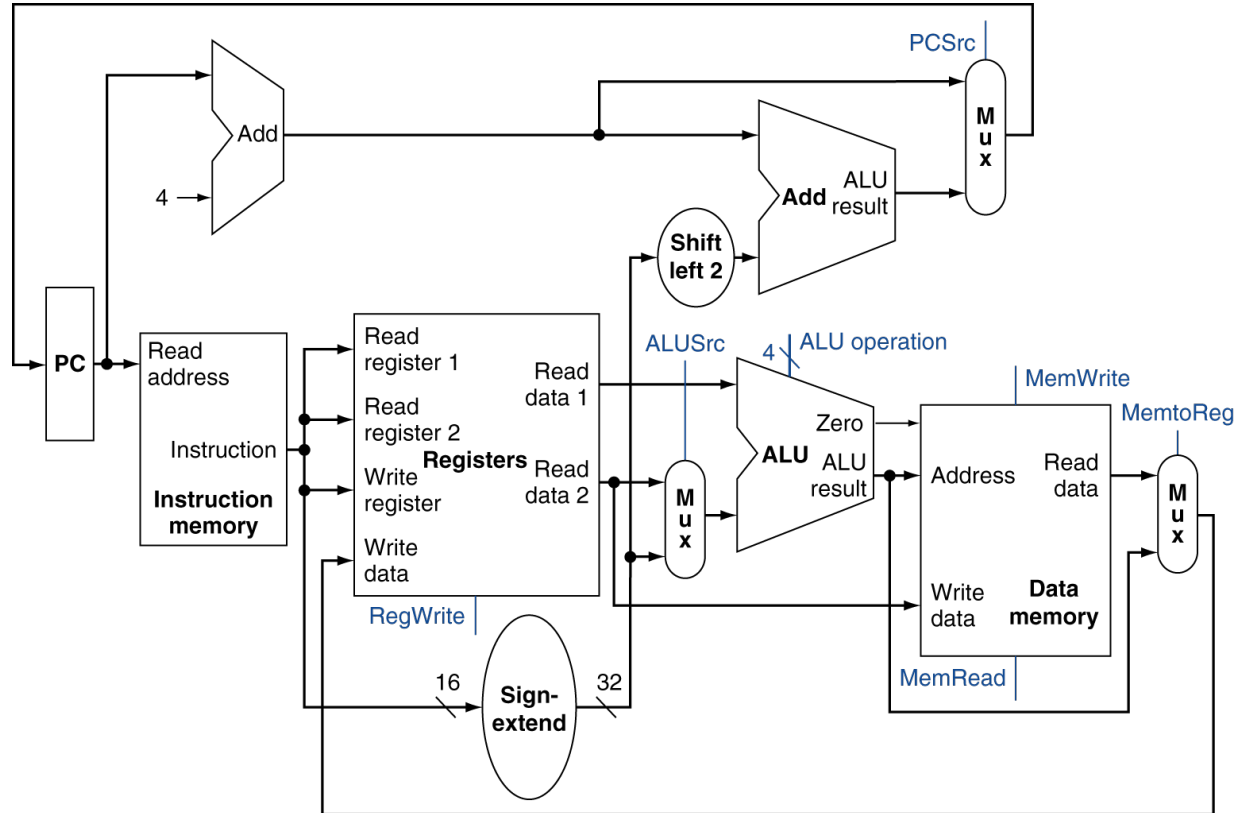
Cache Miss Type

Suppose you experience a cache miss on a block (let's call it block A). You have accessed block A in the past. There have been precisely 1027 different blocks accessed between your last access to block A and your current miss. Your block size is 32-bytes and you have a 64 kB cache (recall a kB = 1024 bytes). What kind of miss was this?

Selection	Cache Miss
A	Compulsory
B	Capacity
C	Conflict
D	Both Capacity and Conflict
E	None of the above

CACHE PERFORMANCE

I-cache vs D-cache



- Separate caches for instruction memory and data memory
- I-cache: instruction cache
- D-cache: data cache

Measuring Cache Performance

- Components of CPU time
 - Program execution cycles
 - Includes cache hit time
 - Memory stall cycles
 - Mainly from cache misses
- With simplifying assumptions:
- Miss penalty is the number of cycles the pipeline stalls on a cache miss waiting for the data to arrive from memory (or from the next level of cache)

Memory stall cycles

$$= \frac{\text{Memory accesses}}{\text{Program}} \times \text{Miss rate} \times \text{Miss penalty}$$

$$= \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}$$

Cache Miss Cycles Per Instruction

Given

- I-cache miss rate = 2%
- D-cache miss rate = 4%
- Miss penalty = 100 cycles
- Load & stores are 36% of instructions

	I-cache	D-cache
A	$.02 * 100$	$.04 * 100$
B	$.02$	$.04$
C	$.02 * .36 * 100$	$.04 * .36 * 100$
D	$.02 * 100$	$.04 * .36 * 100$

Cache Performance Example

- Given
 - I-cache miss rate = 2%
 - D-cache miss rate = 4%
 - Miss penalty = 100 cycles
 - Base CPI (ideal cache) = 2
 - Load & stores are 36% of instructions
- Miss cycles per instruction
 - I-cache: $0.02 \times 100 = 2$
 - D-cache: $0.36 \times 0.04 \times 100 = 1.44$
- Actual CPI = $2 + 2 + 1.44 = 5.44$

Average Memory Access Time

- Hit time is also important for performance
- Average memory access time (AMAT)
 - $AMAT = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$
- Example
 - hit time = 1 cycle, miss penalty = 20 cycles, l-cache miss rate = 5%
 - AMAT =

Cache Speed Factors

- Memory lookup time
- Hit rate
- Size
- Frequency of collisions

How Much Associativity

- Increased associativity decreases miss rate
 - But with diminishing returns
- Simulation of a system with 64 kB D-cache, 64-byte blocks

Miss rate:

- 1-way: 10.3%
- 2-way: 8.6%
- 4-way: 8.3%
- 8-way: 8.1%

```
For (int i = 0; i < 100000000; i++)
```

```
    sum += A[i];
```

Assume each element of A is 4 bytes and sum is kept in a register. Assume a direct-mapped 32 kB cache with 32 byte blocks. Which changes would help the hit rate of the above code?

Selection	Change
A	Increase to 2-way set associativity
B	Increase block size to 64 bytes
C	Increase cache size to 64 kB
D	A and C combined
E	A, B, and C combined

Performance Summary

- When CPU performance increases
 - The miss penalty becomes more significant
- When we decrease the base CPI
 - A greater proportion of time spent on memory stalls
- When we increase the clock rate
 - Memory stalls account for more CPU cycles
- Can't neglect cache behavior when evaluating system performance

Reading

- Next lecture: More Caches!